

MSIS

Multi-Source Intelligent System

위성·드론 시계열 영상 기반
객체 페어링 변화 탐지 및 GraphRAG 기반 판독보고서 자율 생성 시스템

프로젝트 소개서

한화에어로스페이스 지원 첨부 자료

작성일: 2026년 09월 16일

1. 프로젝트 개요

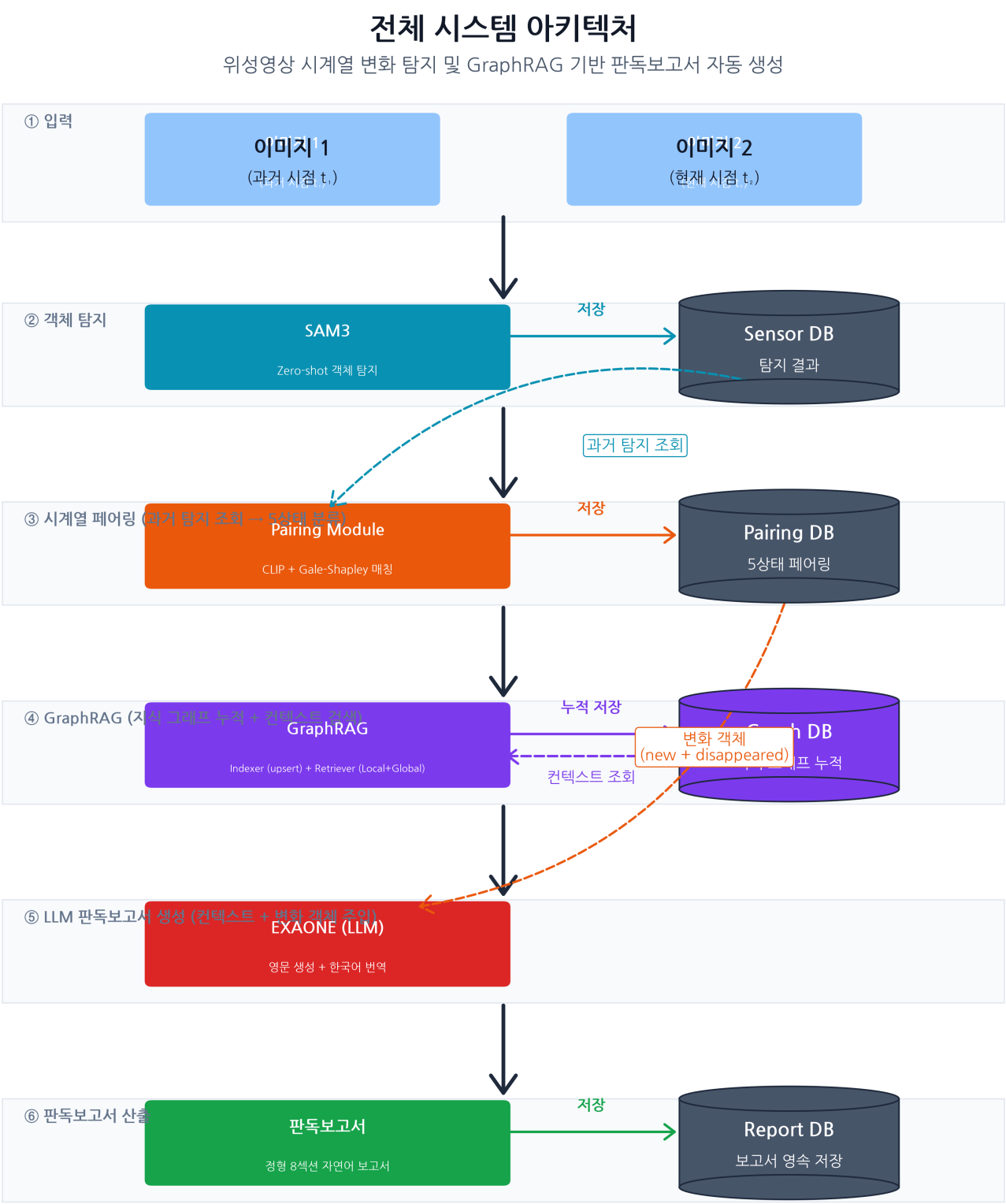
본 프로젝트(MSIS, Multi-Source Intelligent System)는 위성·드론 시계열 영상 한 쌍을 입력받아 변화를 자동 탐지하고, 그 결과를 정형 판독보고서로 자율 생성하는 end-to-end 통합 시스템이다. 종래의 픽셀 단위 패치 격자 비교 방식이 객체 이동을 추적하지 못하고 이진 판정에 그치며 시계열 누적 패턴을 인식하지 못하는 한계, 그리고 LLM 기반 보고서 생성 시 시계열 컨텍스트 부재와 환각 위험 문제를 함께 해결하는 것을 목표로 한다.

시스템은 객체 인스턴스 단위 페어링 기반 변화 탐지와 결정론적 GraphRAG 이력 누적을 결합한 뒤, LangGraph 기반 critic 노드를 갖춘 AI Agent 구조로 판독보고서를 자율 생성한다. 각 파이프라인 단계(탐지 · 페어링 · 그래프 인덱싱 · 보고서 생성 · 번역)의 산출물은 critic 노드에서 자동으로 검증되며, 검증 실패 시 해당 단계로 자동 재실행되거나 사람에게 이관되는 자기 검증 구조를 갖는다. 각 단계의 산출물은 별도 데이터베이스에 세션 식별자와 함께 저장되어 결과의 재현과 검증이 가능하다.

핵심 기여점

- 객체 인스턴스 단위 이원 페어링 — 고정형(위경도 근접 + CLIP)과 이동형($N \times M$ 유사도 행렬 + Gale-Shapley 안정 매칭)으로 분리 처리, 다상태(6종) 정밀 분류
- 결정론적 지식 그래프 누적 — 위경도 소수점 2자리 반올림(약 1km 격자) 결정론 노드 키, AI 호출 없는 upsert, Louvain 커뮤니티 탐지로 반복 관측 패턴 자동 발견
- LangGraph critic 기반 자율 판독보고서 생성 — Local + Global 압축 컨텍스트를 LLM에 주입, 영문 생성 → 한국어 번역 각 단계에 critic 노드를 배치해 환각·형식 오류 자동 검증 및 재실행
- 세션 단위 재현 가능 4-DB 아키텍처 — Sensor/Pairing/Graph/Report DB를 물리적으로 분리, session_id 기반 HITL 재처리 지원

2. 전체 시스템 아키텍처



각 DB는 세션 식별자(session_id)로 묶여 HITL 재처리 시 자동 재계산됩니다.

[도 1] 전체 파이프라인 흐름도

2. 전체 시스템 아키텍처 (설명)

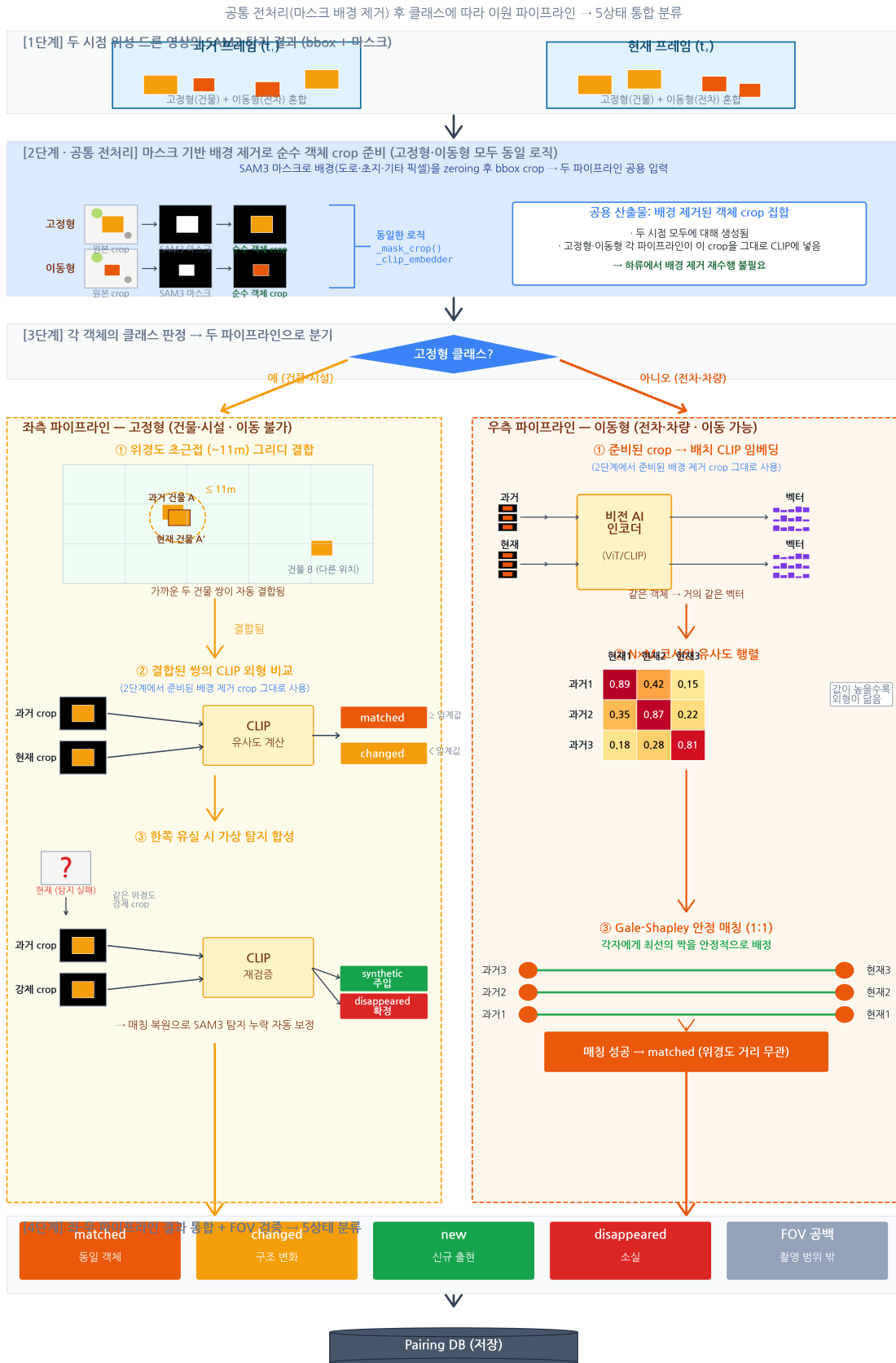
도 1은 두 시점 위성·드론 영상 한 쌍에서 시작해 최종 판독보고서가 나오기까지의 전체 흐름을 보여준다. 시스템의 핵심은 탐지 → 페어링 → 지식 누적 → 보고서 생성의 4단계 파이프라인이다.

각 단계는 LangGraph 노드로 모델링되고, 각 단계 뒤에는 critic 노드가 배치되어 산출물의 품질을 자동 검증한다. 검증을 통과하면 다음 단계로 진행하고, 실패하면 같은 노드로 재실행되거나 임계값 조정 후 재시도되며, 반복 실패 시 HITL(Human-in-the-Loop)로 이관되는 자기 검증 구조이다.

각 단계의 산출물은 별도 데이터베이스(Sensor / Pairing / Graph / Report)에 저장되어 결과를 재현하고 검증할 수 있으며, 특정 단계의 결과가 사용자에게 의해 수정되면 후속 단계가 자동 재계산되는 구조를 갖는다.

3. 객체 페어링 변화 탐지 상세 (이원 처리)

도 2. 객체 페어링 변화 탐지 상세



[도 2] 페어링 모듈 — 공통 전처리 + 이원 파이프라인

3. 객체 페어링 변화 탐지 상세 (설명)

도 2는 두 시점 영상의 객체를 어떻게 짝지어 변화를 잡아내는지를 상세히 보여준다. 핵심 아이디어는 **공통 전처리 후 객체 성질에 따른 이원화**이다. 두 시점 영상은 먼저 SAM3 기반 zero-shot 객체 탐지를 거쳐 각 객체가 마스크와 함께 검출되고, 마스크 기반 배경 제거로 순수 객체 crop이 생성된다. 이 crop은 이후 고정형·이동형 두 파이프라인이 공용으로 사용한다.

고정형 객체 처리

건물·시설 같은 **고정형** 객체는 위경도 초근접(~11m) 그리디로 결합한 뒤 결합된 쌍의 CLIP 외형 비교로 matched / changed 여부를 판정한다. 한쪽 시점에서 탐지가 유실된 경우 같은 위경도로 강제 crop을 생성해 CLIP 재검증으로 synthetic detection을 주입함으로써 SAM3 탐지 누락을 자동 보정한다.

이동형 객체 처리

전차·차량 같은 **이동형** 객체는 위치가 바뀔 수 있으므로 배치 단위 CLIP 임베딩의 $N \times M$ 코사인 유사도 행렬을 만들고 **Gale-Shapley 안정 매칭**을 적용해 짝을 찾는다. Gale-Shapley는 서로 바꿔치기 하고 싶은 짝이 남지 않는 안정된 1:1 매칭을 보장하며 그리디 방식의 중복·교차 페어링을 원천 차단한다.

두 갈래 결과는 하나의 다상태(matched / changed / new / disappeared / 촬영공백 2종)로 통합 분류되어 Pairing DB에 저장된다.

4. GraphRAG 기반 시공간 이력 누적

도 3. GraphRAG 기반 시공간 이력 누적 및 압축 컨텍스트 생성

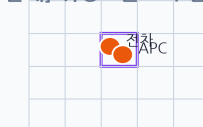
페어링 결과를 AI 호출 없이 지식 그래프에 누적하고, 관련 이력을 압축해 LLM에 전달

[1단계] Pairing DB로부터 최근 회차 비교의 결과 배치를 조회 (한 회차 = 두 시점 영상 1쌍의 변화 탐지 결과)

예: 3회차 비교(2025.04.10 ↔ 08.20)에서 나온 페어링 레코드 배치

회차	클래스	상태	과거 좌표	현재 좌표	신뢰도
3회차	tank	matched	(37.5765, 126.9680)	(37.5765, 126.9680)	0.91
3회차	APC	new	-	(37.5771, 126.9698)	0.79
3회차	building	disappeared	(37.5750, 126.9660)	-	0.87

[2단계] 위 정보를 소수점 2자리로 반올림 (≈ 1 km 격자 단위) → 결정론적 노드 키 생성



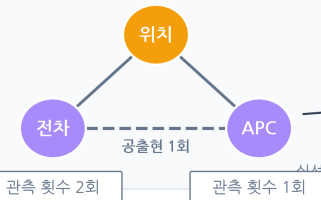
격자화
(소수점 2자리)

loc:37.58,126.97
asset:tank:37.58,126.97
asset:APC:37.58,126.97

← 위치 노드
← (전차 × 격자) 노드
← (APC × 격자) 노드

[3단계] Before (1~2회차 누적)

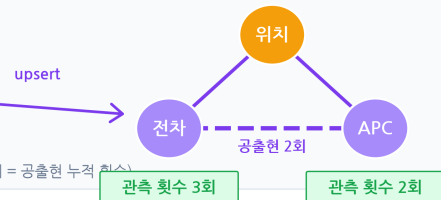
전차·APC 이미 같은 격자 관측 이력 존재



3회차 upsert (지금)
2025.04.10 ↔ 08.20
전차 · APC 같은 격자 재관측
→ 관측 횟수 +1, 공출현 엣지 +1

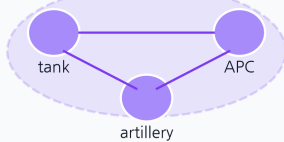
After (3회차 반영)

관측 횟수와 공출현 엣지 가중치 모두 +1

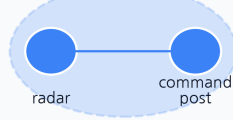


군집 1: 기갑 복합체

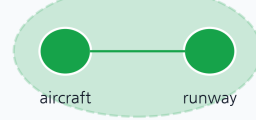
[4단계] Louvain 군집화 — 자주 함께 등장하는 자산들을 자동 그룹화



군집 2: C2 인프라



군집 3: 항공 작전 거점



→ 자주 함께 등장한 자산들을 그래프 알고리즘이 자동으로 그룹화 (AI 호출 없음)

[5단계] 지식 그래프 영속 저장

Graph DB (노드 + 엣지 + 군집)

[6단계] 보고서 생성 시 관련 이력 검색

Local Search

대상 지역 반경 R 내
자산 이력·카운터 조회

Global Search

관련 군집 요약
(doctrine 패턴)

~500 토큰 압축 컨텍스트

"기갑 복합체 8회 반복 관측,
지난 30일간 3회 재배치..."
→ LLM 프롬프트에 prepend

[도 3] GraphRAG 인덱싱 및 압축 컨텍스트 생성

4. GraphRAG 기반 시공간 이력 누적 (설명)

도 3은 매 회차 페어링 결과를 어떻게 시간축으로 쌓아 재활용 가능한 지식으로 만드는지를 보여준다. 핵심은 위경도를 소수점 2자리로 반올림(약 1 km 격자)해 **결정론적 노드 키**(예: loc:37.58,126.97 · asset:tank:37.58,126.97)를 만들고, 같은 (자산 × 격자) 조합이 반복 관측될 때마다 관측 횟수와 공출현 엣지 가중치를 +1씩 누적하는 upsert 방식이다.

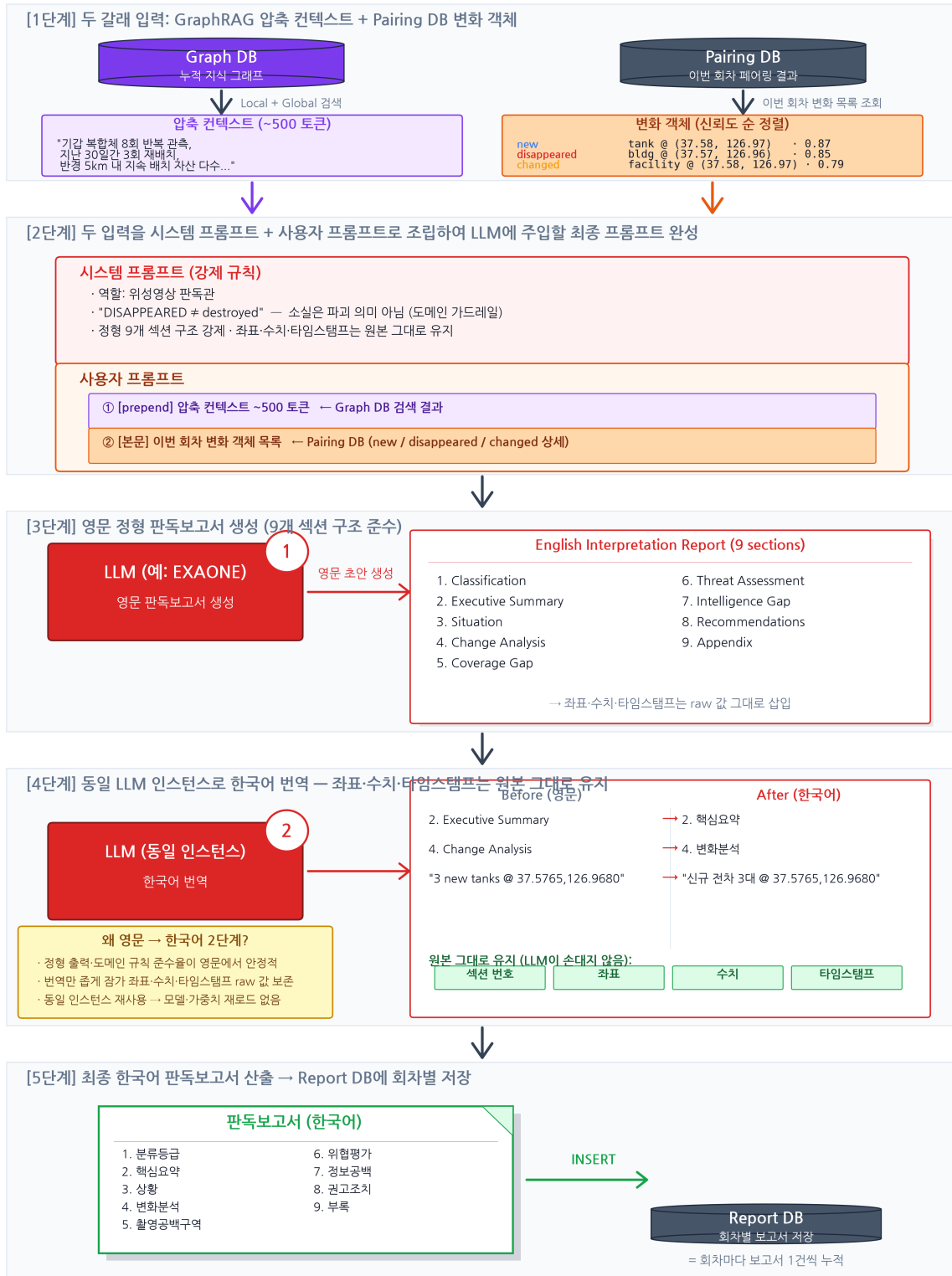
이 upsert 과정에는 **LLM 호출이 없어** 동일 입력에 대해 항상 동일한 그래프가 재현되며, 비용도 발생하지 않는다. 누적된 그래프에는 **Louvain 커뮤니티 탐지**가 적용되어 자주 함께 관측되는 자산 군집이 자동으로 발견된다.

보고서 생성 시에는 대상 지역 반경의 자산 이력을 조회하는 Local Search와 관련 커뮤니티 요약물 가져오는 Global Search를 병행하여, 관련 이력만 **약 500 토큰**의 압축 컨텍스트로 반환한다. 이 압축 컨텍스트가 다음 단계의 LLM 프롬프트에 주입되어 판독보고서의 시공간 맥락을 채운다.

5. LangGraph Critic 기반 판독보고서 자율 생성

도 4. 판독보고서 자율 생성

그래프 압축 컨텍스트 + 이번 회차 변화 객체를 LLM에 주입 → 영문 정형 보고서 → 무오염 한국어 번역



[도 4] 판독보고서 생성 파이프라인

5. LangGraph Critic 기반 판독보고서 자율 생성 (설명)

도 4는 이번 회차 변화(팩트)와 과거 누적 맥락(배경)을 결합해 어떻게 완성도 있는 판독보고서를 만드는지를 보여준다. 두 입력의 역할이 다르다. **Pairing DB**는 이번 회차에 무엇이 변했는지의 팩트를 제공하고, **Graph DB**는 그 변화가 이례적인지 반복 패턴인지의 맥락을 제공한다. 두 소스를 시스템/사용자 프롬프트로 조립해 LLM에 넣으면 정형 9개 섹션(분류등급·핵심요약·상황·변화분석·촬영공백구역·위협평가·정보공백·권고조치·부록) 보고서가 생성된다.

Report Critic

LangGraph 파이프라인에서는 각 생성 단계 뒤에 **critic 노드**가 배치된다. **Report Critic**은 9개 섹션이 모두 존재하는지, 보고서에 인용된 자산·좌표가 실제 입력 페어링 레코드에 존재하는지, 'DISAPPEARED ≠ destroyed' 같은 도메인 가드레일이 준수되었는지를 검증한다. 검증 실패 시 실패 사유를 피드백으로 담아 재생성한다.

Translation Critic

이어 동일 LLM 인스턴스가 한국어로 번역하되 좌표·수치·타임스탬프는 원본 그대로 보존하며, **Translation Critic**이 섹션 구조·raw 값 보존·영문 잔재 여부를 검증한다. 두 단계의 critic 검증을 모두 통과한 최종 한국어 보고서가 Report DB에 세션별로 저장된다.

이 자기 검증 구조로 인해 환각 발생 가능성이 실질적으로 낮아지고, 좌표·수치 등 결정적 raw 값의 원본 보존이 보장된다. Critic이 여러 번 재시도해도 통과하지 못하는 케이스는 HITL로 자동 이관되어 사람이 최종 확인·수정할 수 있다.

6. LangGraph Critic 노드 아키텍처

전체 파이프라인은 LangGraph의 상태 머신으로 모델링된다. 각 생성 단계(generator 노드) 뒤에는 critic 노드가 배치되어 산출물의 품질을 자동 검증하며, 검증 결과에 따라 다음 단계로의 라우팅 · 같은 노드로의 재실행 · HITL(Human-in-the-Loop) 이관이 조건부로 결정된다. 총 5개의 critic 노드가 탐지 → 페어링 → 그래프 인덱싱 → 보고서 생성 → 번역 각 단계를 검증한다.

Critic 노드	검증 항목	실패 시 처리
① Detection Critic (SAM3 탐지 검증)	· 클래스·confidence 분포가 정상 범위인가 · degenerate bbox(0 크기)가 있는가 · 이미지 특성 대비 탐지 수가 비정상은 아닌가	SAM3 프롬프트 재조정 또는 confidence 임계값 완화 후 재실행
② Pairing Critic (이원 페어링 검증)	· matched : new : disappeared 비율의 급격한 편향 여부 · CLIP 유사도 분포가 임계값 근처에 몰려있진 않은가 · Gale-Shapley 후 미매칭 자산이 과다한가	SIMILARITY_MATCH_THRESHOLD 조정 후 재실행 또는 HITL 이관
③ Graph Critic (GraphRAG 인덱싱 검증)	· 격자 셀당 노드 수가 극단적으로 편중되지 않았는가 · Louvain 커뮤니티 개수·크기가 정상 범위인가 · upsert 후 관측 횟수·엣지 가중치가 실제로 반영됐는가	_LOC_PRECISION(격자 해상도) 조정 후 재인덱싱 · 경고만 남
④ Report Critic (영문 보고서 검증)	· 정형 9개 섹션 헤더가 모두 존재하는가 · 보고서에 인용된 자산·좌표·수치가 실제 입력 페어링 레코드에 존재하는가 (팩트 매칭) · 'DISAPPEARED ≠ destroyed' 등 도메인 가드레일 준수 여부	실패 사유를 프롬프트 피드백에 추가하여 재생성 (self-refine I
⑤ Translation Critic (한국어 번역 검증)	· 9 섹션 번호·헤더가 그대로 유지됐는가 · 좌표·수치·타임스탬프가 raw 값 그대로 보존됐는가 (정규식 diff) · 영문 잔재가 없는가 (언어 일관성)	재번역 요청 · 반복 실패 시 HITL 이관

파이프라인 상태(State) 관리

각 노드 간에는 LangGraph의 상태 객체(PipelineState)가 전달된다. 이 객체에는 세션 식별자, 각 단계의 산출물(detections · pairings · graph_context · english_report · korean_report), 각 critic의 통과/실패 이력(critic_history), 그리고 무한 루프 방지를 위한 단계별 재시도 카운터(retry_counts)가 포함된다.

재시도 정책과 HITL 이관

- 각 critic 노드는 최대 재시도 횟수(예: 3회)를 상태 객체로 관리하여 무한 루프를 방지한다
- 재시도가 소진되어도 검증에 통과하지 못하면 HITL 이관 노드로 라우팅되어 웹 UI를 통해 사람이 결과를 확인하고 수동 수정할 수 있다
- HITL에서 수정된 결과는 해당 session_id의 후속 DB 레코드를 자동 재계산하도록 다음 단계로 다시 흘러보낸다
- Critic은 대부분 규칙 기반(rule-based)으로 구현하여 결정론성과 재현성을 유지하고, LLM 기반 critic은 최소로 제한하여 GraphRAG의 결정론 강점을 훼손하지 않도록 설계한다

7. 데이터 아키텍처 (4개 DB)

각 파이프라인 단계의 산출물은 물리적으로 분리된 4개의 SQLite 데이터베이스에 세션 식별자(session_id)와 함께 저장된다. 이 구조는 (i) 결과의 재현·검증, (ii) 특정 단계의 부분 재실행, (iii) HITL 재처리 시 후속 단계 자동 재계산을 가능하게 한다.

7.1 Sensor DB — 촬영 영상과 SAM3 탐지 결과

image_records (이미지 메타)

컬럼	타입	저장 내용
id	UUID	이미지 고유 식별자
capture_time	DateTime	원본 촬영 시각
source_type	String	"satellite" 또는 "drone"
image_path	Text	이미지 파일 경로
lat_center / lon_center	Float	이미지 중심 좌표
lat/lon_min, lat/lon_max	Float	지리 bbox (픽셀→위경도 변환 기준)
resolution_m	Float	GSD (미터/픽셀)
sensor_platform	String	예: WorldView-3, MQ-9
det_width / det_height	Integer	탐지에 사용된 해상도
session_id	UUID	파이프라인 세션 식별자

detection_records (SAM3 탐지 결과)

컬럼	타입	저장 내용
id / image_id	UUID	탐지 식별자, 부모 이미지 FK
detection_time	DateTime	탐지 시각 (=capture_time)
object_class, object_class_index	String, Int	클래스명 및 인덱스
confidence	Float	SAM3 confidence
bbox_x1 / y1 / x2 / y2	Float	픽셀 단위 bbox
lat / lon	Float	픽셀→위경도 변환 결과
mask_rle	Text	RLE 인코딩된 SAM3 마스크
mask_area_px	Float	마스크 픽셀 수
source_type / extra / session_id	String, JSON, UUID	소스·부가정보·세션

7.2 Pairing DB — 두 시점 페어링 결과

각 회차 비교(과거 프레임 vs 현재 프레임)의 페어링 결과를 저장한다. 상태별로 채워지는 필드가 다르다: matched/changed는 양쪽 다, new는 current만, disappeared는 past만, past_not_included/current_not_included는 한쪽만 채워진다.

컬럼	타입	저장 내용
id, pairing_time	UUID, DateTime	레코드 식별자·페어링 시각
lat_center, lon_center	Float	지역 중심 좌표
status	String	matched changed new disappeared past_not_included current_not_included
current_detection_id	UUID	현재 탐지 FK (Sensor DB)
current_object_class, current_confidence	String, Float	현재 클래스·신뢰도
current_lat, current_lon	Float	현재 위경도
current_capture_time, current_bbox	DateTime, JSON	현재 시각·bbox
past_detection_id	UUID	과거 탐지 FK (Sensor DB)
past_object_class, past_confidence	String, Float	과거 클래스·신뢰도
past_lat, past_lon	Float	과거 위경도
past_capture_time, past_bbox	DateTime, JSON	과거 시각·bbox
source_type, session_id	String, UUID	소스·세션 식별자

7.3 Graph DB — 결정론적 지식 그래프

위경도 격자 반올림으로 만든 결정론적 노드 키를 사용해 자산·위치·관계를 upsert 방식으로 누적한다. AI 호출 없이 동일 입력에 동일 그래프가 재현되며, Louvain 알고리즘으로 반복 관측 패턴이 커뮤니티 레코드로 저장된다.

graph_entities (노드)

컬럼	타입	저장 내용
id (PK)	String	결정론적 노드 키 (loc:37.58,126.97 · asset:tank:37.58,126.97)
entity_type	String	location asset
name	String	표시 이름
properties	JSON	location: {lat, lon} · asset: {object_class, lat, lon, new_count, matched_count, changed_count}
first_seen / last_seen	DateTime	최초·최근 관측 시각
observation_count	Integer	누적 관측 횟수

graph_relations (엣지)

컬럼	타입	저장 내용
id	UUID	엣지 식별자
source_id / target_id	String	연결된 entity id
relation_type	String	found_at (자산→위치) co_occurred_with (자산↔자산)
properties	JSON	found_at: {count, confidence_sum, first_seen, last_seen, ...} · co_occurred_with: {count, confidence_sum, first_seen, last_seen, ...}
created_at / updated_at	DateTime	생성·갱신 시각

graph_communities (Louvain 군집)

컬럼	타입	저장 내용
id, community_index	UUID, Int	커뮤니티 식별자·번호
label	String	예: Cluster-0: military tank, APC, artillery
member_ids	JSON	소속 자산 노드 id 리스트
member_summary	Text	결정론적 통계 요약 (LLM 호출 없이 생성)
summary	Text	선택적 LLM 요약
created_at	DateTime	생성 시각

7.4 Report DB — 최종 판독보고서

critic 노드 검증을 통과한 최종 한국어 판독보고서를 세션별로 저장한다.

컬럼	타입	저장 내용
id, saved_time	UUID, DateTime	레코드 식별자·DB 저장 시각
report_time	DateTime	LLM 생성 완료 시각
session_id	UUID	파이프라인 세션 식별자
llm_model	String	예: EXAONE-Deep-32B
llm_backend	String	vllm huggingface ollama
pairing_count	Integer	분석에 쓰인 페어링 레코드 수
file_path	Text	.txt 파일 경로 (옵션)
report_content	Text	한국어 9섹션 판독보고서 전문 (분류등급·핵심요약·상황·변화분석·촬영공백구역·위협평가·정

공통 특성

- **세션 격리**: 모든 테이블의 session_id로 세션 단위 조회·삭제·재실행
- **노드 키의 결정론성**: 같은 (자산, 격자) 조합이면 항상 같은 id → upsert 병합 자동
- **재현·검증 가능**: 각 단계 결과를 독립 조회할 수 있어 결과의 재현과 검증이 가능
- **부분 재계산**: HITL로 특정 단계 결과가 수정되면 후속 DB의 해당 session_id 레코드가 재계산

8. 사용 기술 스택

영역	사용 기술	역할
객체 탐지	SAM3 (Segment Anything Model 3)	사용자 지정 클래스의 zero-shot 인스턴스 탐지 (bbox + 마스크)
좌표 변환	pixel_to_geo() 함수	이미지 지리 bbox 선형 보간으로 픽셀 → 위경도
시각 임베딩	CLIP (ViT 기반)	마스크 배경 제거 후 crop → 코사인 유사도 비교용 임베딩
매칭 알고리즘	Gale-Shapley 안정 매칭	이동형 객체의 N×M 유사도 행렬에서 1:1 안정 매칭
지식 그래프	SQLAlchemy + NetworkX	결정론적 노드 키 기반 upsert · Louvain 커뮤니티 탐지
Agent 오케스트레이션	LangGraph	각 단계 노드 + critic 노드 · 조건부 라우팅 · 자기 검증 파이프라인
LLM	EXAONE (LG AI Research)	정형 9섹션 판독보고서 영문 생성 및 한국어 번역
LLM 서빙	vLLM	FastAPI와 동일 프로세스에서 EXAONE 모델 서빙
Backend	FastAPI + Uvicorn (Python)	REST API 서버 · 파이프라인 실행 · HITL 재처리 엔드포인트
Frontend	Vanilla JavaScript + HTML	React·Vue·Streamlit 미사용 · 경량 브라우저 UI
지도 시각화	Leaflet.js + OpenStreetMap	탐지 결과·페어링 상태의 지도 오버레이 시각화
데이터 저장	SQLite (4개 DB 분리)	Sensor / Pairing / Graph / Report DB · session_id 기반 격리

맺음말

본 프로젝트는 위성·드론 시계열 영상 분석에서 **정확한 변화 탐지와 맥락 있는 자율 판독**이라는 두 축을 하나의 end-to-end 파이프라인으로 통합하려는 시도이다. 특히 LangGraph critic 노드 기반의 자기 검증 구조를 도입함으로써 LLM 기반 시스템의 고질적 문제인 환각 위험을 자동으로 억제하고, 결정론적 GraphRAG 이력 누적을 통해 세션이 반복될수록 시스템의 판독 품질이 축적되도록 설계하였다. 이러한 자기 검증형 자율 판독 파이프라인은 위성 정보 판독·감시 경찰·재난 대응 등 실시간성과 신뢰성이 동시에 요구되는 분야에 폭넓게 적용될 수 있다.